

Data Science Internship
at
BigSiBucks Innovation

INTERNSHIP REPORT

Submitted by

VENISH A (312424104256)

in partial fulfillment for the award of the degree of

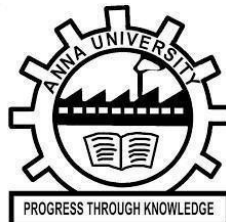
BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING



St. JOSEPH'S INSTITUTE OF TECHNOLOGY
(An Autonomous Institution)



ANNA UNIVERSITY :: CHENNAI 600025

INTERNSHIP COMPLETION CERTIFICATE



BIG-SI-BUCKS
INNOVATION PRIVATE LIMITED
CIN : U27400TN2024PTC169823

30 JUNE 2026
Chennai

CERTIFICATE OF INTERNSHIP COMPLETION

Dear **Venish A**

Subject: Internship Completion Letter – **DATA SCIENCE**

This is to certify that you have successfully completed your internship at **Big Bucks Innovation** as a **DATA SCIENCE** Intern during the period from 05-06-2026 to 30-06-2026 in OFFLINE MODE.

Throughout the internship, you have shown commendable enthusiasm, professionalism, and a sincere approach toward learning and task execution. You were actively involved in the assigned projects and demonstrated a solid understanding of concepts.

Your contributions have been valuable, and we appreciate the dedication

With Best Regards

Mr. Balaji Arumugam
Founder & CEO



+91 8667858430



bigbucksinnovation.com



bigbucksinnovation@gmail.com



No.14 , Prabhu Nagar , Neelankarai Link Road , Thoraipakkam , Chennai - 97

RECOGNISED BY



This is to certify that VENISH A, Register Number 312424104256, has successfully completed an internship at Big Bucks Innovation during the period 3-June-2026 to 30-June-2026, working on the project "Cybersecurity Threat Detection System."

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Big Bucks Innovation for giving me the opportunity to undergo my internship and work on a live project involving cybersecurity and machine learning. This experience allowed me to apply my academic knowledge to a practical, industry-relevant problem and gain valuable hands-on exposure to backend development and system integration.

I am deeply thankful to my Company Mentor, [Company Mentor Name], for the continuous guidance, technical mentorship, and constructive feedback provided throughout the internship. I also sincerely thank the Head of the Department and my Faculty Guide, [Faculty Guide Name], at St. Joseph's College of Engineering for their encouragement, academic support, and valuable suggestions during the course of this internship.

I would also like to thank my teammates in Team AutoBots – Vishnu Prasaath S, Sabareesh V, and Vasanth P - for their cooperation, collaborative spirit, and shared commitment, which made the successful completion of the Cybersecurity Threat Detection System possible.

Finally, I am grateful to my parents, family, and friends for their constant encouragement and support throughout this internship journey.

VENISH A

CONTENTS

S.No.	Section	Page No.
1	Introduction	5
2	Company Profile	7
3	Internship Activities	9
4	Learning Outcomes	13
5	Challenges & Suggestions	16
6	Conclusion	18
7	References	19
8	Annexure	20

1. INTRODUCTION

1.1 Objectives of the Internship

- To gain practical, industry-relevant exposure to real-world software development beyond the academic curriculum.
- To understand how machine learning models are integrated into a production-style backend application.
- To develop hands-on skills in backend engineering, REST API design, and database integration using FastAPI and SQLite.
- To experience working as part of a cross-functional, team-based software development environment.
- To study the design and working of a Cybersecurity Threat Detection System that classifies network activity as normal or malicious.
- To apply classroom knowledge of programming, databases, and machine learning to a live, industry-defined problem statement.

1.2 Significance of the Internship

Every organization that runs a computer network faces a constant risk of cyberattacks. Employees use company networks daily for email communication, file sharing, and access to online business applications, and each of these activities creates an opportunity for attackers to attempt phishing, malware injection, unauthorized access, or network intrusion. As network traffic grows in volume and complexity, manually monitoring every connection for suspicious behaviour becomes impractical, making automated and intelligent threat detection systems essential for modern network security.

This internship provided an opportunity to work on exactly such a system - a Cybersecurity Threat Detection System - as part of a four-member team, AutoBots, at Big Bucks Innovation. The significance of the internship lies not only in the technical exposure it provided to machine learning, backend engineering, and system integration, but also in the professional exposure to real project timelines, team coordination, mentor reviews, and the discipline of building software that other team members' work depends on.

Working on a live, production-style problem - rather than an isolated academic assignment - helped bridge the gap between theoretical concepts learned in the classroom and their practical application in an industry setting. The internship also strengthened soft skills such as communication, collaborative problem-solving, and time management, which are equally important to technical competence in a professional software development environment.

1.3 Scope of the Report

The scope of this report is limited to describing the internship experience, the activities undertaken as Backend Developer in Team AutoBots, and the learning outcomes derived from the Cybersecurity Threat Detection System project completed at Big Bucks Innovation. It documents the objectives, the internship environment,

the tools and technologies used, the tasks performed on a week-by-week basis, the underlying system architecture, and the personal and professional skills developed over the course of the internship.

The report is organized into eight chapters: an introduction outlining the objectives, significance, and scope of the internship; a profile of the host organization; a detailed account of the internship activities including the weekly plan, tasks performed, and system workflow; the learning outcomes and skills acquired; the challenges encountered along with suggestions for future interns; a concluding chapter summarizing the overall experience; a list of references; and an annexure containing the day-wise internship diary, a sample API interaction, and a glossary of technical terms used throughout the project.

2. COMPANY PROFILE

2.1 Company Overview

Big Bucks Innovation is a technology solutions company that works on data-driven and AI-powered products for clients across multiple industries. The company focuses on building practical software solutions in areas such as data analytics, machine learning, automation, and secure application development, and it regularly takes on interns to work alongside its engineering teams on live, real-world projects.

As part of its internship programme, Big Bucks Innovation assigns interns to small project teams that are expected to independently research, design, and build a complete solution to a defined problem statement, under the guidance of an assigned company mentor. This structure gives interns exposure to the full software development lifecycle - from requirement analysis and design through implementation, testing, and presentation of results.

2.2 Vision and Mission

Vision: Big Bucks Innovation aims to be a trusted technology partner that helps organizations make better decisions and build safer, smarter systems through the practical application of data science, machine learning, and modern software engineering practices.

Mission: The company's mission is to deliver reliable, well-engineered, and innovative technology solutions, while nurturing talent through hands-on internship and training programmes that bridge the gap between academic learning and industry practice.

2.3 Products / Services and Core Activities

- Data Analytics and Business Intelligence
- Machine Learning and Artificial Intelligence solutions
- Cybersecurity and Threat Monitoring Solutions
- Web and API-based Application Development
- Automation and Workflow Optimization
- Cloud-based Software Deployment

2.4 Internship Environment

The internship at Big Bucks Innovation was organized as a team-based project, with each intern in Team AutoBots responsible for a specific module of the Cybersecurity Threat Detection System. Daily stand-ups and periodic reviews with the company mentor ensured that individual modules stayed aligned and could be integrated smoothly. This environment gave practical exposure to backend engineering, collaborative software development, version control, and the discipline of building software that other team members' work depends on.

2.5 Organizational Structure

Big Bucks Innovation follows a flat, project-oriented organizational structure typical of a growing technology solutions company. At the top, the Leadership Team sets overall strategic direction and manages client relationships. Below this sit functional divisions - including the AI & Data Analytics division under which Team AutoBots operated - each headed by a Project Lead / Company Mentor who supervises a small number of engineers and interns working on live, client-facing or internal products.

- Leadership Team
- Engineering and Product Development
- AI and Data Analytics Division (Team AutoBots)
- Quality Assurance and Client Delivery
- Human Resources and Operations

This lean structure allowed for quick decision-making, direct mentor access, and a short feedback loop between interns and senior engineers, which was particularly valuable when resolving integration issues during the project.

2.6 Achievements and Focus Areas

Although a young company, Big Bucks Innovation has built a reputation for delivering functional proof-of-concept and pilot applications quickly, particularly in the areas of predictive analytics and applied machine learning. The company places a strong emphasis on mentoring interns and junior engineers through real project ownership rather than shadow assignments, and several intern-built prototypes have gone on to inform full-scale internal tooling. This intern-driven innovation culture was directly reflected in the Cybersecurity Threat Detection System project, which was conceived, designed, and largely implemented by the four-member AutoBots team under mentor supervision.

3. INTERNSHIP ACTIVITIES

3.1 Project Undertaken

Detail	Description
Project Title	Cybersecurity Threat Detection System
Team Name	AutoBots
Team Members	Vishnu Prasaath S, Venish A, Sabareesh V, Vasan P
My Role	Backend Developer
Duration	[From Date] to [To Date] (25 working days)
Departments Visited	Engineering / Product Development team, under the AI & Data Analytics division

3.2 Machines / Software Used

Category	Tools / Software
Programming Language	Python 3.11
IDE	VS Code
Backend Framework	FastAPI, Uvicorn
Database	SQLite, SQLAlchemy, DB Browser for SQLite
Machine Learning Libraries	Scikit-learn, Pandas, NumPy
Frontend	Streamlit
Version Control	Git and GitHub
Testing	Postman
Design / Documentation	draw.io, Google Docs, Google Sheets

3.3 Weekly Activities

The 25-day internship was organized into five broad phases, summarized week by week below. A detailed day-by-day account is maintained in the internship diary submitted separately / attached in the Annexure.

Week	Focus Area	Key Tasks Performed
Week 1 (Day 1–5)	Orientation & Planning	Company induction, team formation, domain research on cybersecurity threats, development environment setup, study of the eight ML algorithms used in the project, requirement analysis.
Week 2 (Day 6–10)	Architecture & Foundation	Designed the system architecture (User → Preprocessing → Threat Detection → Automated Response → Results), set up the FastAPI project structure, designed REST API endpoints and the SQLite database schema, implemented database models.
Week 3 (Day 11–15)	Core Backend Development	Built CRUD APIs for network activity data, integrated data-preprocessing scripts, tested endpoints with Postman, coordinated with the ML team, and wrapped the classification models (Logistic Regression, KNN, Decision Tree, Random Forest) for API use.
Week 4 (Day 16–20)	Model Integration	Built the unified threat-detection endpoint, integrated K-Means clustering, integrated PCA-based feature reduction, built the automated alert/response endpoint, and integrated the Reinforcement Learning response agent.
Week 5 (Day 21–25)	Integration, Testing & Delivery	Connected the Streamlit frontend to the backend, performed end-to-end integration testing, fixed bugs and optimized performance, documented the backend APIs, and completed final testing and the project presentation.

3.4 Tasks Performed (Detailed)

- Designed the overall system architecture and data-flow workflow for the project.
- Developed the FastAPI backend application and its REST API endpoints (/preprocess, /activity, /detect-threat, /cluster, /respond).
- Integrated the SQLite database, including the network_activity, predictions, and response_logs tables, using SQLAlchemy.
- Implemented model-loading utilities and wrapper functions to serve Linear Regression, Logistic Regression, KNN, Decision Tree, Random Forest, K-Means, PCA, and a Reinforcement Learning agent through a consistent API interface.
- Coordinated with teammates to align data formats between preprocessing, model inference, and the frontend.

- Performed backend-side testing of API endpoints using Postman, and fixed data-validation and performance issues.
- Connected the Streamlit dashboard to the backend and supported end-to-end integration testing.
- Documented the backend architecture and API usage using FastAPI's Swagger UI and a supplementary written guide.

3.5 Project Work - System Workflow

At a high level, the completed application follows a five-stage workflow: the User submits network activity data, the system performs Data Preprocessing on that data, the preprocessed data passes through Threat Detection models (classification, clustering, and traffic prediction), an Automated Response module decides on and triggers an appropriate defensive action using a Reinforcement Learning agent, and finally the Results and Alerts are returned to the user through the Streamlit dashboard.

3.6 Database Schema Design

The backend used a relational schema built with SQLAlchemy on top of SQLite, consisting of three primary tables that map directly to the stages of the detection workflow. Foreign-key relationships link a submitted activity record to its resulting prediction, and each prediction to any automated response that was triggered, allowing the full lifecycle of a single network event to be traced from ingestion to resolution.

Table	Key Fields	Purpose
network_activity	id, source_ip, destination_ip, protocol, packet_size, timestamp	Stores raw and preprocessed network traffic records submitted for analysis
predictions	id, activity_id (FK), model_used, prediction_label, confidence_score, created_at	Stores the classification/clustering output generated for each activity record
response_logs	id, prediction_id (FK), action_taken, agent_state, reward, executed_at	Stores the automated response action selected by the reinforcement-learning agent and its outcome

3.7 REST API Endpoint Reference

The eight endpoints listed below formed the documented API contract shared with the frontend and machine-learning teammates. Each endpoint accepts and returns JSON, and was documented using FastAPI's automatically generated Swagger UI in addition to a short internal usage guide.

Method	Endpoint	Description
POST	/activity	Accepts and stores a new network activity record
POST	/preprocess	Cleans and transforms raw activity data for model input
POST	/detect-threat	Runs classification models and returns a threat label with confidence
POST	/cluster	Runs K-Means clustering to group activity by behavioural similarity

Method	Endpoint	Description
GET	/predictions/{id}	Retrieves the stored prediction for a given activity record
POST	/respond	Triggers the reinforcement-learning response agent for a flagged threat
GET	/response_logs	Retrieves the history of automated responses taken
GET	/health	Simple health-check endpoint used during integration testing

3.8 Machine Learning Models - Purpose and Integration

As Backend Developer, my responsibility was not to train these models but to build the wrapper functions and API layer that loaded each trained model artifact and served its predictions through a consistent interface, regardless of the underlying algorithm.

Model	Type	Role in the System
Linear Regression	Regression	Predicted expected network traffic volume for baseline comparison
Logistic Regression	Classification	Baseline binary classifier distinguishing normal from malicious traffic
K-Nearest Neighbours	Classification	Distance-based classifier used for comparison against Logistic Regression
Decision Tree	Classification	Interpretable rule-based classifier used to validate feature importance
Random Forest	Classification (ensemble)	Primary high-accuracy classifier used for the final threat label
K-Means Clustering	Unsupervised	Grouped traffic into behavioural clusters to flag anomalous patterns
PCA	Dimensionality Reduction	Reduced feature dimensionality before clustering and heavier classifiers
Reinforcement Learning Agent	Sequential decision-making	Selected an appropriate automated response action based on the detected threat state

4. LEARNING OUTCOMES

4.1 Engineering Concepts Applied

- Software architecture design - designing a modular client-server architecture connecting a frontend, backend, ML models, and a database.
- Data structures and algorithms - applied while designing efficient preprocessing and feature-handling logic for real-time API requests.
- Database design principles - normalization, foreign-key relationships, and schema design applied to the `network_activity`, `predictions`, and `response_logs` tables.
- Applied machine learning concepts - regression, classification, clustering, dimensionality reduction (PCA), and reinforcement learning, and how each is served through a live application rather than a notebook.
- Software engineering practices - version control with Git, API contract design, modular code organization, and systematic testing.

4.2 Software Learned

Software / Tool	Purpose Learned
FastAPI & Uvicorn	Building and serving REST API endpoints for a live backend service
SQLAlchemy & SQLite	Object-relational mapping and relational database integration
Scikit-learn	Loading, wrapping, and serving trained machine learning models
Streamlit	Understanding frontend-backend integration for a data dashboard
Postman	API testing, validation, and contract verification
Git & GitHub	Version control and collaborative development across a team
draw.io	System architecture and workflow diagramming

4.3 Equipment Handled

As this was a software-focused internship, the primary "equipment" handled was a personal development workstation configured with the required software stack (Python 3.11, VS Code, Git). No specialized hardware or laboratory equipment was used; all work was carried out through standard software development tools described above.

4.4 Industrial Practices Observed

- Agile-style daily stand-ups and periodic mentor reviews to track progress and resolve blockers early.
- Use of a shared, version-controlled repository with a consistent branching and commit convention across the team.
- Maintaining a documented API contract so that frontend, backend, and machine learning work could proceed in parallel without conflicts.
- Structured testing practices - unit testing of individual endpoints followed by full end-to-end integration testing before delivery.
- Code and design documentation intended for maintainability and handover, rather than only for personal reference.
- Professional communication and coordination across a cross-functional team working on interdependent modules.

4.5 Personal Reflection

This internship gave me my first real experience of building the backend for a system that depends on machine learning outputs rather than static business logic alone. Working closely with teammates who were training the models pushed me to think carefully about how to design an API that would remain stable even as the underlying models changed - a lesson I expect to carry into future backend engineering work. It also strengthened my confidence in taking ownership of a significant technical component within a larger, team-driven project.

4.6 Skills Acquired - Summary

Skill Category	Specific Skill	Proficiency Gained
Technical	REST API design with FastAPI	Working proficiency
Technical	Relational database design (SQLAlchemy / SQLite)	Working proficiency
Technical	Serving machine learning models via an API	Working proficiency
Technical	API testing and contract verification with Postman	Working proficiency
Technical	Git-based collaborative version control	Working proficiency
Professional	Cross-functional team coordination	Developing
Professional	Technical documentation	Developing
Professional	Time management under fixed deadlines	Developing

4.7 Comparison with Academic Learning

Much of the theoretical foundation for this internship - data structures, relational database concepts, and the mathematics underlying regression and classification algorithms - had already been covered in coursework such as Database Management Systems, Machine Learning, and Software Engineering. What the internship

added was the practical discipline of applying these concepts under real constraints: designing an API that other people's code depends on, handling malformed or unexpected input, and keeping a system stable while multiple components evolve in parallel. This gap between knowing an algorithm and being able to serve it reliably inside a live application was one of the most valuable takeaways from the internship.

5. CHALLENGES & SUGGESTIONS

5.1 Challenges Faced

- Aligning data formats between the machine learning models, developed independently by teammates, and the backend API contract.
- Serving multiple different model types (regression, classification, clustering, and reinforcement learning) through a single consistent API interface.
- Debugging integration issues that only appeared when the frontend, backend, and models were run together rather than in isolation.
- Managing database schema changes as new fields were needed once the automated response module was integrated.
- Balancing backend response time against the computational cost of running several machine learning models per request.
- Coordinating work and testing schedules across a four-member team working on interdependent modules.
- An intermittent SQLite database-locking issue that appeared only under near-simultaneous requests, making it harder to reproduce and fix.

5.2 How the Challenges Were Overcome

Most integration challenges were resolved by agreeing on a shared, documented JSON schema for requests and responses early in the project and revisiting it whenever a new model was added. Performance concerns were partly addressed by applying PCA before the heavier classification models and by caching model objects in memory rather than reloading them on every request. Regular short sync-ups with teammates helped catch data-format mismatches early, before they could affect full integration. The database-locking issue was resolved by reviewing and adjusting how concurrent write operations were handled by the ORM layer.

5.3 Suggestions for Future Interns

- Agree on a documented API contract (request/response formats) with all teammates as early as possible - this saves significant integration time later.
- Set up version control, a shared requirements file, and a consistent project structure from day one to avoid environment-mismatch issues.
- Test each module in isolation before attempting full end-to-end integration; this makes it far easier to trace the source of a bug.
- Maintain a simple bug/issue tracker (even a shared spreadsheet) from the start of development rather than only during testing weeks.
- Keep a daily work log - it makes writing the final internship report significantly easier and helps track progress against the plan.
- Don't hesitate to ask the company mentor for clarification early; small ambiguities in the problem statement are much cheaper to resolve at the start than after implementation.

5.4 Key Takeaways

Beyond the specific technical fixes described above, the recurring theme across most challenges faced during this internship was the importance of agreeing on interfaces - between modules, between teammates, and between the system and its users - before implementation begins. Nearly every integration issue traced back to an assumption that was never explicitly written down and confirmed with the relevant teammate. This is a lesson that extends well beyond this particular project and one I intend to apply deliberately in future team-based software development work.

6. CONCLUSION

The internship at Big Bucks Innovation provided valuable, hands-on experience in building a real-world Cybersecurity Threat Detection System as part of Team AutoBots. As Backend Developer, I designed the system architecture, built the FastAPI backend, integrated the SQLite database, and connected multiple machine learning models - covering traffic prediction, threat classification, behavioural clustering, dimensionality reduction, and automated response - into a single, working application accessible through a Streamlit dashboard.

Beyond the technical deliverable, the internship offered a valuable overall experience of working within a professional, team-based software development environment - coordinating with teammates on interdependent modules, responding to mentor feedback, and delivering a working system within a fixed timeline. This industrial exposure complemented my academic learning and gave me a much clearer picture of how software is actually built, tested, and delivered in industry.

Looking ahead, this internship has strengthened my interest in backend engineering and the integration of machine learning into real applications, and the skills gained - API design, database integration, and cross-functional collaboration - are directly relevant to my future career goals in software development.

7. REFERENCES

1. Scikit-learn Documentation - Machine Learning in Python, scikit-learn developers.
2. FastAPI Official Documentation - Modern, fast web framework for building APIs with Python.
3. Streamlit Official Documentation - A framework for building data applications in Python.
4. SQLite Documentation - Self-contained, serverless SQL database engine.
5. Pandas and NumPy Documentation - Python libraries for data manipulation and numerical computing.
6. Course and internship material provided by Big Bucks Innovation on cybersecurity fundamentals and threat detection.
7. AutoBots Team Internship Project Presentation - Cybersecurity Threat Detection System (internal project document).

8. ANNEXURE

8.1 Annexure A - Day-wise Internship Diary

The table below summarizes the activities performed on each working day of the 25-day internship, consolidating the detailed daily log maintained throughout the programme.

Day	Date	Activities Performed
Day 1	03-Jun-2026	Company induction and orientation session; introduction to Big Bucks Innovation's teams and policies.
Day 2	04-Jun-2026	Team AutoBots formed; roles assigned - Backend Developer, ML Engineers, Frontend Developer.
Day 3	05-Jun-2026	Domain research on common cybersecurity threats and network intrusion patterns.
Day 4	06-Jun-2026	Development environment setup - Python 3.11, VS Code, Git, virtual environment configuration.
Day 5	08-Jun-2026	Study of the eight machine learning algorithms shortlisted for the project.
Day 6	09-Jun-2026	Requirement analysis and problem-statement discussion with the Company Mentor.
Day 7	10-Jun-2026	Designed the overall system architecture: User -> Preprocessing -> Threat Detection -> Response -> Results.
Day 8	11-Jun-2026	Set up the FastAPI project structure and folder organization.
Day 9	12-Jun-2026	Designed REST API endpoint list and request/response JSON contract.
Day 10	13-Jun-2026	Designed the SQLite database schema and entity relationships.
Day 11	15-Jun-2026	Implemented SQLAlchemy database models for network_activity table.
Day 12	16-Jun-2026	Implemented SQLAlchemy database models for predictions and response_logs tables.
Day 13	17-Jun-2026	Built CRUD APIs for submitting and retrieving network activity data.
Day 14	18-Jun-2026	Integrated data-preprocessing scripts into the /preprocess endpoint.
Day 15	19-Jun-2026	Tested initial endpoints using Postman and fixed request-validation issues.
Day 16	20-Jun-2026	Coordinated with the ML team on trained-model file formats and loading conventions.
Day 17	22-Jun-2026	Wrapped Logistic Regression and KNN classification models for API use.
Day 18	23-Jun-2026	Wrapped Decision Tree and Random Forest classification models for API use.
Day 19	24-Jun-2026	Built the unified /detect-threat endpoint combining all classification models.
Day 20	25-Jun-2026	Integrated K-Means clustering and PCA-based feature reduction into the pipeline.
Day 21	26-Jun-2026	Built the automated alert/response endpoint (/respond).

Day	Date	Activities Performed
Day 22	27-Jun-2026	Integrated the Reinforcement Learning response agent with the /respond endpoint.
Day 23	29-Jun-2026	Connected the Streamlit frontend to the backend; began end-to-end integration testing.
Day 24	30-Jun-2026	Fixed integration bugs, optimized response time, and documented backend APIs.
Day 25	01-Jul-2026	Final testing, bug fixes, documentation review, and project presentation to the mentor.

8.2 Annexure B - Sample API Request and Response

The following illustrates a representative request sent to the /detect-threat endpoint and the corresponding JSON response returned by the backend.

Sample Request (POST /detect-threat):

```
{ "source_ip": "192.168.1.14", "destination_ip": "10.0.0.5", "protocol": "TCP", "packet_size": 1450,
  "timestamp": "2026-06-24T10:32:15" }
```

Sample Response:

```
{ "activity_id": 1042, "model_used": "Random Forest", "prediction_label": "malicious",
  "confidence_score": 0.94, "action_taken": "block_ip" }
```

8.3 Annexure C - Glossary of Technical Terms

Term	Definition
API	Application Programming Interface - a defined way for two software components to communicate.
REST	An architectural style for designing networked applications using standard HTTP methods.
JSON	JavaScript Object Notation - a lightweight, text-based data-interchange format.
FastAPI	A modern Python web framework used to build the project's REST API backend.
ORM	Object-Relational Mapping - a technique for mapping database tables to programming objects.
CRUD	Create, Read, Update, Delete - the four basic operations of persistent storage.
PCA	Principal Component Analysis - a technique for reducing the number of features in a dataset.
K-Means Clustering	An unsupervised algorithm that groups data points into a fixed number of clusters.
Reinforcement Learning	A machine learning approach in which an agent learns actions by receiving rewards.
Confusion Matrix	A table used to evaluate the performance of a classification model.
Swagger UI	An automatically generated interactive interface for exploring and testing an API.

Term	Definition
Endpoint	A specific URL path at which an API exposes a particular piece of functionality.